# Patricia26 API Reference

***Release 1.5.0***

**Gene C**

**Oct 13, 2026**

# PATRICIA26 REFERENCE


**1 patricia26** **1**
- 1.1 Overview . . . 1
- 1.2 Patricia26 Python class . . . 2
- 1.3 Performance . . . 3
- 1.4 Documentation . . . 4
- 1.5 AI Tools . . . 5
- 1.6 License . . . 5

**2 Python Particia26 Class: API Reference** **7**
- 2.1 Python API . . . 8

**3 Particia26 C-Library: API Reference** **13**
- 3.1 Data Structures . . . 13
- 3.2 Functions . . . 14

**4 Migrating from PyTricia** **19**

**Python Module Index** **21**

CHAPTER

ONE

# PATRICIA26

## 1.1 Overview

A Patricia tree is designed to map some data to a network. A network is a range of consecutive IP addresses. Since an IP address is a number, this provides a way to attach data to a range of numbers.

For example, it can be used to associate a geographic location, a country for example, with an IP network.

**patricia26** implements this as a compiled library and provides a Python module that uses it.

patricia26 has two components: A Patricia Trie library and a Python module using the library. It provides:

- a modern, C23 compliant (Linux) version of the the original C-code.
- a Python module generated using Cython that uses the shared c-library.
- an autogenerated API reference guide for both the C and Python code.

The project name made unique to allow the C library to be installed alongside the original and to avoid any conflict with other similar Python modules such as PyTricia.

This project was forked from Dave Plonka's Patricia Library along with some modifications of the core c-code by Joel Sommers' PyTricia. The main change introduced in PyTricia changed *node->prefix* from *prefix_t ** to a *prefix_t* thereby reducing the number of *malloc* calls.

The C-code has been tidied some and is now C23 compliant. See the *meson.build* file for more details on compile and link options.

The Python module, *Patricia26*, uses the C library *libpatricia16.so*. The shared library must be installed and available for the Python module to run.

Using Cython to generate the Python module keeps the module highly performant. Uusing a shared patricia library allows for:

- C and Python applications to use the same library.
- the core patricia library to be built with stringent compile and link options.

The Cython generated C-code is built using slightly different options than the core c-clibrary since manually tidying Cython generated c-code is not practical. Consequently, we use less stringent compile options on the Cython code.

While this is working well for our own use cases, should any issues arise, please let us know.

The documentation including the C and Python API reference is in patricia26.pdf in the *Docs* directory and is also installed in *usr/share/patricia26/*

The Python module, *Patricia26* offers similar functionality to *PyTricia* Both modules are very fast and are built on top of the same original core Patricia C-code. We have used *PyTricia* and found it to be robust and works well.

*Patricia26* can be somewhat faster, especially for bulk and cidr lookups.

The API is a little different, but changing from one to the other is quite straightforward. Please see *Migrating from PyTricia*.

### 1.1.1 Documentation

The manual provides detailed information and is available in both HTML and PDF formats. Both are installed under */usr/share/cidrtools/docs*.

The manual is also available at: readthedocs.

### 1.1.2 Additional Python Approach

There is a tentative Python module, *Patricia_int*, that uses integers / *ipaddress* instead of IP and CIDR strings. However, this version is significantly slower, probably due to Python not using simple 32 and 128 bit number representations that can be mappped directly to C, thus necessitating additional conversion overhead that inflicts a pretty big performance penalty. Consequently, we're not using this class and so it has not had the usage and testing that *Patricia26* class has.

## 1.2 Patricia26 Python class

The Python module wraps the core patricia C-functions. This is a short highlight on using Pytricia26. Please see the documentation section with the Reference API guide which provides details on the class and its member functions.

**Instantiate Class**

The class internally keeps both a 32 bit IPv4 tree and a 128 bit IPv6 tree, so instantiating for either is done the same way.

```
from patricia26 import Patricia26
pyt = Patricia26()
```

**Saving Values**

Values are inserted into the tree either one at a time or from a list of tuples.

For example:

```
pyt[ip_or_cidr] = value
```

or

```
pyt.insert(ip_or_cidr, value)
```

or

```
items = [(ip_or_cidr_1, value_1), ((ip_or_cidr_2, value_2), ... ]
pyt.bulk_insert(items)
```

**Looking up Values**

The best way to lookup the value associated with the longest prefix match (LPM) is to use the optimized lookup() method.

The dictionary subscript syntax (*pyt[ip_or_cidr]* can also be used for both CIDRs or IP addresses.

When looking up a list of items, *bulk_lookup(list_of_ip_or_cidrs)* is much faster than looping over pyt.lookup(). It returns a list of values.

When the value and it's associated LPM prefix are needed, then *lookup_lpm()* returns a tuple of (lpm, value).

Examples:

```
value = pyt.lookup(ip_or_cidr)
value = pyt[ip_or_cidr]
(lpm, value) = pyt.lookup_lpm(ip_or_cidr)
```

with ip_or_cidr ~ '10.0.0.2' or '10.1.0.0/22'

```
results = pyt.bulk_lookup(cidrs)
```

is much faster than

```
results = []
for cidr in cidr_list:
    results.append(pyt.lookup(cidr))
```

## 1.3 Performance

The benchmark code is available in the *tests* directory. We test lookups on both IP and CIDR inputs using bulk_lookup() as well as looping on lookup(). Similarly to retrieve both the lpm and it's associated value.

Please view these test results as a rough guide only. We recognize there may be ways to improve performance that we may not have taken advantage of.

Doing lookups on plain IP adresses is a bit faster than on CIDR blocks since the latter involves some prefix length calculations. Bulk lookups are substantially faster than using a loop.

Bulk IP lookups can be two to three times as fast as doing the same in a loop: 68 million per second versus 23 million per second. Bulk CIDR lookups at 65 million per second compared to 25 million per sec using a loop.

Bulk lookups should be used with care if the size of the input list is very large due to potential memory pressure. The benchmark tests worked well with 5 million and 50 million IP/CIDRs iterations, with the performance remaining roughly constant. There is variability with different number of iterations but the overall picture remains clea. Tests were run on a laptop with 32 GB of memory.

We should write some additional benchmarks using larger trees, but the high level picture is that these modules are both pretty decent.

Looking up the pair of (LPM prefix, value) using looup_lpm() is a lot slower than retrieving the value alone (8.5 million per second). So, if the lpm prefix is not needed, its far better to use lookup() or bulk_lookup().

*Pytricia*s performance doing value lookups is similar to looping in *Patricia26* and of course slower than bulk_lookup().

The tests lookup the same IP / CIDR repeatedly which is obviously not terribly realistic. So with that in mind this is sample bencmark output. All benchmarks run with 50 million iterations:

```
          Patricia26 (50,000,000 iterations)
  Method    Input       Lookup    Seconds  Lookups / sec
    Bulk       IP          val    0.7300    68,489,951
    Bulk     CIDR          val    0.7627    65,552,597
    Loop       IP          val    2.1655    23,089,855
    Loop     CIDR          val    1.9984    25,019,973
    Loop     CIDR   (lpm, val)    5.6163     8,902,680
    Loop       IP   (lpm, val)    5.5182     9,060,900


           PyTricia (10,000,000 iterations)
  Method    Input       Lookup    Seconds  Lookups / sec
-------- -------- ------------ ---------- --------------
   [key]       IP          val    2.0406    24,502,376
     get       IP          val    3.2581    15,346,357
     get     CIDR          val    3.6105    13,848,587
    getk     CIDR   (lpm, val)    9.3112     5,369,895
    getk       IP   (lpm, val)    8.9636     5,578,110
```

The Patricia26 tests use pyt.lookup(), pyt.bulk_lookup() and pyt.lookup_lpm(). The lpm lookups carry a burden of additionally providing the prefix string which shows degraded performance compared to simply looking up the value.

The PyTricia tests use pyt[ip] and pyt.get(cidr) and retrieve the (lpm, value) ~ (lpm = pyt.get_key(), pyt[lpm])

### 1.3.1 Pickling: Compact File Size

When Python uses pickle to save a Patricia26, the class pickles the actual data of (prefix, value) pairs in the tree. The *glue* modes are automatically re-created by the c-clibrary when unpickling and are not saved in the pickled file. This leads to smaller pickled files, and the savings can be very significant. Smaller files naturally means faster IO.

For example, for a tree with 355,951 IPv4 prefixes and their associated values the pickled file size is approximatrly 4MB; if the glue nodes are kept the size would be about 25 MB - so file size is around 16% of what it would be had we retained all the glue nodes.

## 1.4 Documentation

The package includes a pre-built PDF guide in *Docs/patricia26.pdf* The PDF and HTML version can be built by running:

```
make latexpdf ; make latexpdf
make html
```

Building the docs requires *sphinx* and *hawkmoth*. The documentation includes an auto-generated API Reference Guide that covers both the Patricia C-library and the Patricia26 Python class.

The PDF documentation is installed in *usr/share/patricia26*

## 1.5 AI Tools

Anthropic's Claude played a significant role updating and checking all C-code.

Google's Gemini was super helpful with building the cython code.

## 1.6 License

The original patricia code by Dave Plonka is licensed under the University of Michigan and Merit Network. Modifications and additions are licensed by Gene C under the Lesser Gnu Public License version 3 or later.

Modifications made by Joel Sommers <jsommers@colgate.edu> in PyTricia are licensed under the Lesser Gnu Public License version 3 or later.

The license files are available in the *src/share* directory and are installed in the usual place.

CHAPTER

TWO

# PYTHON PARTICIA26 CLASS: API REFERENCE

This section contains the high-level Python API wrapper for the Patricia Trie. The package includes API stubs file allowing tools such as *mypy* to type check code that uses Patricia26.

Here are some of the important methods:

- Patricia26()
  Instantiate class
- Store Value
  pyt[cidr] = value pyt.bulk_insert(cidr_values)
  cidr_values is a list of tuples[cidr, value]
- lookup()
  returns the value associated with the input prefix or None if no match Can also use dictionary lookup:
  value = pyt.lookup(cidr) value = pyt[cidr]
- bulk_lookup()
  similar to lookup() but takes a list of prefixes and returns a list of values. Faster than looping on lookup()
  values = pyt.bulk_lookup(cidrs)
- lookup_lpm()
  similar to lookup() but returns a tuple of the (lpm, value). Faster than looping on get_prefix() plus lookup() by maybe 15% - 20%.
- get_prefix()
  Returns the LPM prefix associated with IP or CIDR string.
- prefixes(), prefixes_v4, prefixes_v6
  returns a list of all active prefixes in the tree. The _v4 / _v6 variants return list of IPv4 / IPv6 prefixes.
- freeze() / thaw()
  lock/unlock the tree when modifying it.
- parent()
  Returrns the prefix that is the immediate parent of the lpm.
- children()
  returns a list of prefixes that are subnets of the lpm prefix
- num_prefixes_v4(), num_prefixes_v6()
  Returns the number of *active* IPv4 or IPv6 prefixes in the corresponding tree. Note that len(pyt) will return the total number of prefixes across both IPv4 and IPv6 trees.
  Note that this count includes the active nodes which includes *glue* nodes that patricia automatically inserts when needed, not just those explicitly added.

- dump_to_file() / load_from_file()
  Writes or reads a class instance to or from a file stram object.

Supports the *contains* method.

When checking if prefix is in the tree the *in* operator returns True if the prefix is either an exact match of a tree prefix or is a subnet of one. In contrast, the has_prefix() method checks for an exact match to an existing prefix.

There are couple of pytricia compatibility methods that may be useful:

- keys() - an alias for prefixes().
- get_key() - an alias for get_prefix()

For example, Adding and removing key/values to/from the Patricia tree is done using:

```
pyt = Patricia26()
key = '10.0.0.0/24'
val = 'lan-A'
pyt[key] = val
del pyt[key]
```

The next section has the auto-generated API reference.

## 2.1 Python API

**class** patricia26.**Patricia26**

Bases: object

**bulk_insert**(*items: Iterable[tuple[str, Any]]*) → None

Insert a list of tuples of (prefix, value) into the tree:

**Parameters**

**items** – list of (prefix, tuples) to add to the tree

**bulk_lookup**(*ip_strings: list[str]*) → list[Any]

Similar to lookup() but takes a list list of IP or CIDR strings as input.

Significantly faster than looping on lookup() as it uses optimied C code.

**Parameters**

**ip_strings** – A list of target IP or CIDR strings.

**Returns**

A list of values that match each corresponding ip_string in input If an ip_string has no matching prefix, then None is returned for that element.

**children**(*prefix_str: str*) → list[str]

Extracts a list of all child subnets (those with smaller prefix than the parent)

**Parameters**

**prefix_str** – The prefix to lookup

**Returns**
A list of more specific subnets that sit beneath the LPM(prefix_str).

**dump_to_file**(*file_obj: IO[bytes]*)
Write the tree to the file_object stream.

**Parameters**
**file_obj** – File object that was opened in binary mode ('wb')

**freeze**() → None
Locks both ipv4 and ipv6 trees prevent updates or changes

**get_key**(*prefix_str: str*) → str | None
Alias for get_prefix()

**get_prefix**(*prefix_str: str*) → str | None
Finds and returns the prefix (CIDR) string for an IP or CIDR.

**Parameters**
**prefix_str** – The IP/CIDR string to check.

**Returns**
The closest enclosing network block CIDR string, or None if unmatched.

**has_key**(*ip_str: str*) → bool
Alias of has_prefix()

**has_prefix**(*ip_str: str*) → bool
Determines if an exact prefix match exists in the tree.

**Parameters**
**ip_str** – The prefix to check (e.g., "192.168.0.0/16").

**Returns**
True if the prefix exists (non-glue)

**insert**(*prefix_str: str*, *value: Any*) → None
Inserts value to the LPM of an IP or CIDR block string.

Saem as __setitem__.

**Parameters**

- **prefix_str** – The IP subnet string (e.g., "192.0.2.0/24") used to lookup the LPM
- **value** – Custom Python object metadata to associate with this network.

**Raises**
**RuntimeError** – If attempting to modify a frozen Patricia26 tree.

**iter_v4**() → Iterator[str]
Provides an iterator over all IPv4 prefixes

**Yields**
The next network prefix CIDR block strings stored in the tree.

**iter_v6**() → Iterator[str]

Provides an iterator over all IPv6 prefixes

Yields: The next network prefix CIDR block strings stored in the tree.

**keys**() → list[str]

Alias for prefixes()

**load_from_file**(*file_obj: IO[bytes]*)

Read the tree from the file object stream.

**Parameters**
**file_obj** – File object opened in binary mode ('rb')

**lookup**(*ip_str: str*) → Any

Return the value associated with LPM (longest prefix match) matching ip_str

LPM is the longest prefix match.

**Parameters**
**ip_str** – An IP address or CIDR string (e.g., "192.0.2.45" or "10.0.0.0/24").

**Returns**
The custom Python object metadata mapped to the matched network. or None if unmatched

**lookup_lpm**(*ip_str: str*) → tuple[str, Any]

Similar to lookup() but returns a tuple of (lpm, value) longest-prefix match lookup returning a tuple of (matching_cidr, value).

**Parameters**
**ip_str** – The prefix string to lookup (e.g., "192.168.0.0/16").

**Returns**
tuple of (lmp, value) or (None, None) if not found.

**num_prefixes_v4**() → int

Returns the total number of IPv4 prefixes in the tree. :returns: Number of IPv4 prefixes.

**num_prefixes_v6**() → int

Returns the total number of IPv6 prefixes in the tree. :returns: Number of IPv6 prefixes.

**parent**(*prefix_str: str*) → str | None

Find and return the parent prefix that encloses the input prefix.

**Parameters**
**prefix_str** – The prefix to lookup the parent for.

**Returns**
The parent network prefix if found, or None if prefix is not contained within any subnet in tree

**prefixes**() → list[str]

Returns a list of all prefixes :returns: List of all IPv4 and IPv6 prefixes in the tree.

**prefixes_v4**() → list[str]

Returns a list of all IPv4 tree prefixes. :returns: List of all IPv4 prefixes in the tree.

**prefixes_v6**() → list[str]

Returns a list of all IPv6 tree prefixes :returns: List of all IPv6 prefixes in the tree.

**thaw**() → None

Unlocks a frozen tree to re-allow modifications

CHAPTER

THREE

# PARTICIA26 C-LIBRARY: API REFERENCE

## 3.1 Data Structures

struct **prefix_tag**

An IP address plus it's network prefix.

This structure maps structured multi-family properties (IPv4 and IPv6 layouts) into a unified interface. The IP address is held in a union to handle either IPv4 or IPv6.

Dev Note: - Support for IPv6 is no longer #if defined away via HAVE_IPV6 - bitlen is now 1 byte - long enough for ipv4 and ipv6. - Add 1 byte padding to keep struct explicitly aligned on even byte boundary

Attributes:

sa_family_t **family**

Address protocol (AF_INET | AF_INET6 )

uint8_t **bitlen**

netowrk bitmask mask length; aka cidr prefix len (e.g., 24 for a /24 block)

int **ref_count**

Ref couner for allocation tracking with memory reclamation.

union **[anonymous]**

Network address (IPV4 or IPv6)

struct in_addr **sin**

add.sin: struct for 32-bit IPv4 address

struct in6_addr **sin6**

add.sin6 struct for 128-bit IPv6 address

struct **patricia_node_tag**

Core node structure within the Patricia Trie array matrix.

Each node is either a glue node or a node with data payload. Each node is part of the tree.

Follow pytricia in changing node->prefix to be prefix_t instead of prefix_t * simpliefies malloc/free but

Attributes:

uint32_t **bit**

The specific bit index position

prefix_t **prefix**

The prefix for this node

struct *patricia_node_tag* ***l**

Pointer to the left child node

struct *patricia_node_tag* ***r**

Pointer to the right child node

struct *patricia_node_tag* ***parent**

Pointer to the parent node

void ***data**

The data payload for non-glue nodes

struct **patricia_tree_tag**

Top level struct for a Patricia Trie instance.

This is the master anchor for the network tree, Has the head node along with the max number of IP bits, the number of active bodes and a marker whether the tree is *frozen* or not.

Attributes:

patricia_node_t ***head**

Pointer to the top-most tree node

uint32_t **maxbits**

Max number of bits per network IP address

int **num_active_node**

Tracks total number of populated nodes

uint16_t **frozen**

Marks the tree as frozen or not

## 3.2 Functions

static char ***prefix_toa2x_ipv4** (prefix_t *prefix, char *buff, int with_len)

Transpiles a binary network prefix layout into a human-readable string.

Converts structural network prefix parameters back into standard ASCII text string notation representations (e.g., “192.168.1.0/24”).

**Parameters**

- **prefix** – Pointer to the source network prefix block structure.

- **buff** – Pointer to a destination character array buffer. Must be adequately sized to handle string overflows. with_len (int): A boolean flag (1 or 0) toggling whether the subnet mask suffix (like "/24") is appended to the string.

**Returns**

Pointer to the destination string buffer (*buff*) containing the formatted ASCII data layout.

prefix_t ***New_Prefix**(int family, void *dest, int bitlen, prefix_t *prefix)

Constructs a prefix node container from raw network parameters.

This function translates raw binary representations of network addresses into a standardized prefix block layout for the patricia tree.

**Parameters**

- **family** – The network address family protocol (e.g., AF_INET or AF_INET6).
- **dest** – Pointer to the raw binary network address array.
- **bitlen** – The total routing bitmask bitlength.
- **prefix** – Pointer to an allocated destination prefix structure where result will be saved.

**Returns**

Returns 1 on success, 0 if the bitlength exceeds limits, or -1 for unsupported address families.

patricia_tree_t ***New_Patricia**(int maxbits)

Allocates and initializes a new Patricia Trie structure.

This function builds the base routing tree tracking structure, establishing the maximum depth limit of the radix bit-testing operations.

**Parameters**

- **maxbits** – Maximum bitlength allowable for keys in this trie (e.g., 32 for IPv4, 128 for IPv6).

**Returns**

Pointer to the newly allocated and initialized Patricia tree structure, or nullptr if memory allocation fails.

Deallocates all node elements inside the tree matrix while keeping the root intact.

This function walks the entire tree layout structure using an iterative post-order traversal stack. It detaches all child nodes and clears their internal payload properties. Unlike Destroy_Patricia, this function leaves the top-level base tree pointer structure initialized and ready for reuse.

**param patricia**

Pointer to the base Patricia tree object to purge.

**param func**

optional user data deallocation callback function pointer. If provided, the engine will

systematically execute *func(node->data)* on every payload container encountered before destroying the node itself. Pass nullptr if your node data properties do not require custom memory managerment hooks.

void **Destroy_Patricia**(patricia_tree_t *patricia, void_fn_t func)

Deallocates and wipes an entire Patricia Trie system memory layout.

This function performs a recursive post-order tree-walk, freeing every node. It permits an optional custom callback to securely purge node data payloads.

**Parameters**

- **patricia** – Pointer to the Patricia tree system to wipe.

**Func**

An optional custom data-cleanup function pointer to execute on each node's user data value, or nullptr to skip data cleanup.

patricia_node_t ***patricia_search_exact**(patricia_tree_t *patricia, prefix_t *prefix)

Searches the Patricia Trie for an exact prefix match.

This function performs a strict bitwise traversal down the radix tree matrix to locate a node that matches the provided network prefix configuration exactly.

**Parameters**

- **patricia** – Pointer to the base initialized Patricia Trie structure.
- **prefix** – The specific IP prefix mask to query.

**Returns**

Pointer to the matching node structure if found, or nullptr if target prefix does not exist in the tree.

patricia_node_t ***patricia_search_best**(patricia_tree_t *patricia, prefix_t *prefix)

Locates the Longest Prefix Match (LPM) for a given network query.

This function performs an un-mutating longest-match radix search. It maps specific host IP queries back to their broadest covering subnet masks.

**Parameters**

- **patricia** – Pointer to the base Patricia tree structure.

**Prefix**

Pointer to the target prefix/IP to look up.

**Returns**

Pointer to the most specific matching node covering the target address, or nullptr if not found

Searches for a network prefix or inserts it if it is missing.

This is the primary mutating driver of the radix trie. It traverses the bit index paths; if an exact match is missing, a new payload node is created.

**param patricia**
Pointer to the base Patricia tree structure.

**prefix**
Pointer to the target network prefix layout.

**returns**
Pointer to the newly created or pre-existing matching node within the tree matrix, or nullptr if allocation fails.

Extricates an isolated target node container from the Patricia tree network.

Removes a specific node structural point, rearranging internal branch pointer paths to preserve radix trie sorting and lookup logic.

**param patricia**
Pointer to the parent Patricia tree structure.

**param node**
Pointer to the specific node container to remove from the tree array layout.

CHAPTER

FOUR

# MIGRATING FROM PYTRICIA

The `Partcia26` API is a little different and this provides a quick overview how to migrate from `PyTricia`.

### Instantiate Class

```
from pytricia import Pytricia
pyt_ipv4 = Pytricia(32)
pyt_ipv6 = Pytricia(128)
```

becomes:

```
from patricia26 import Patricia26
pyt_ipv4 = Patricia26()
pyt_ipv6 = Patricia26()
```

Note that there is no bit length argument because Patricia26 keeps separate IPv4 and IPv6 trees internally.

### Store Value

```
pyt[ip_or_cidr] = value
```

becomes:

```
pyt[ip_or_cidr] = value

or
pyt.insert(ip_or_cidr, value)

or
cidr_values = [(ip_or_cidr1, value1), (ip_or_cidr2, value2), ...]
pyt.bulk_insert(cidr_values)
```

### Lookup Value

```
val = pyt[ip]
val = pyt.get(cidr)
```

becomes

```
val = pyt.lookup(ip_or_cidr)

or
val = pyt[ip_or_cidr]
```

(continues on next page)

(continued from previous page)

```
or
vals = pyt.bulk_lookup(ip_or_cidr_list)
```

### Lookup LPM and Value

```
lpm = pyt.get_key(ip_or_cidr)
val = pyt[lpm]
```

becomes

```
(lpm, val) = pyt.lookup_lpm(ip_or_cidr)

or
lpm = pyt.get_prefix((ip_or_cidr)
val = pyt.lookup(lpm)
```

# PYTHON MODULE INDEX

p
patricia26, 8